CS 2005: Database System

Project: “Smart Disaster Response MIS with Secure Transactions & Role-Based Control”

Repo Link: https://github.com/maryamfatimaaaaa/disaster_mis.git

Submitted By:
Maryam Fatima 23i3007
Arsal Taimoori 23i0016
Sana Idrees 23i2039

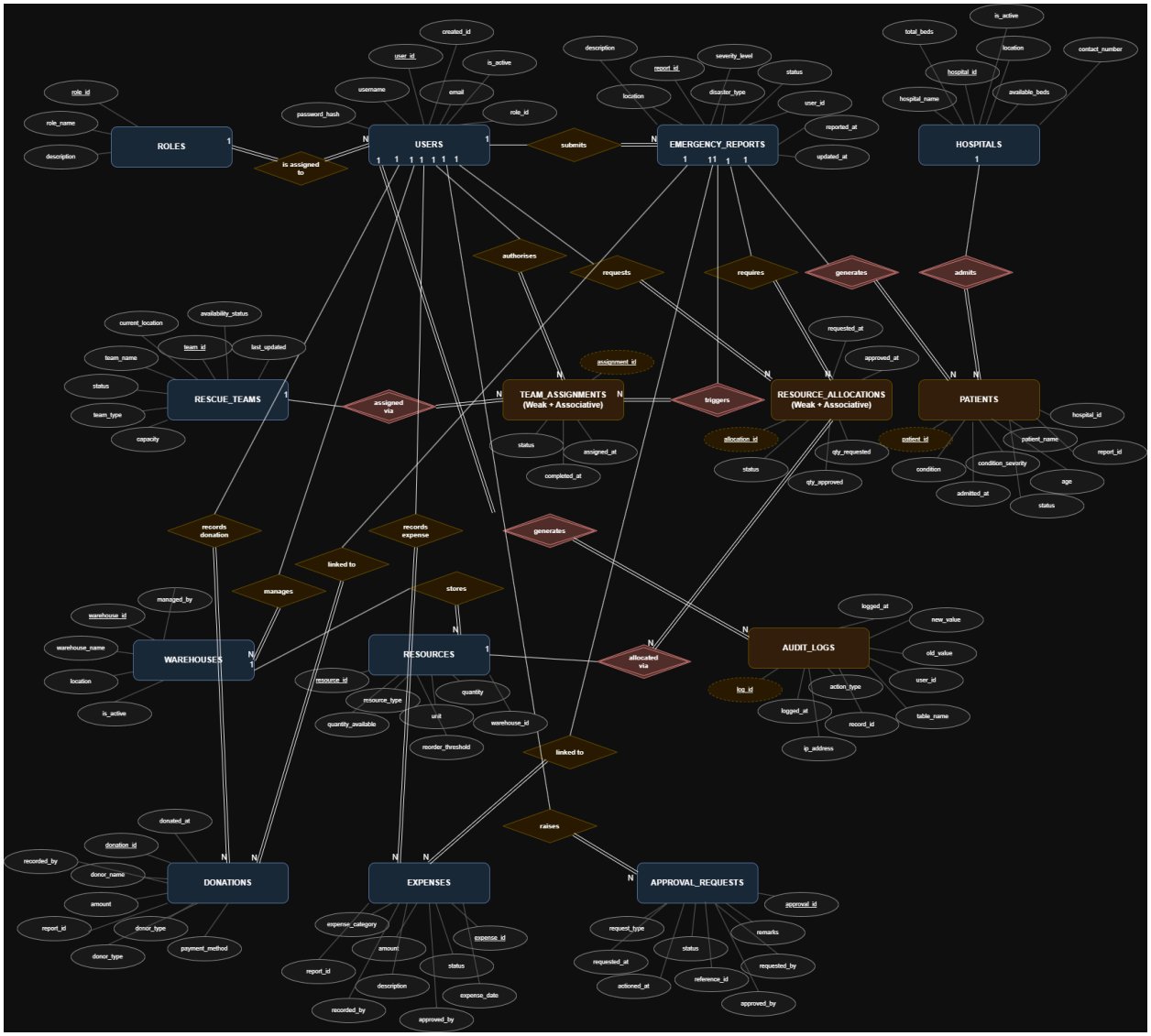
Submitted To:
Ma’am Zoya Sumbul

Table of Contents

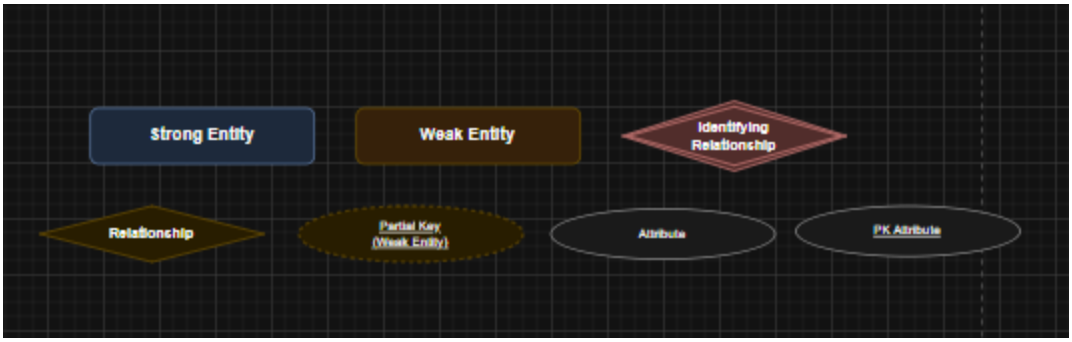
Part 1: Entity Relation Diagram + Normalization

1. ERD

Drive Link:
https://drive.google.com/file/d/11PZRzQ32lB9vJZEsqWXI2O7UJvroKFE8/view?usp=drive_link



Key:



2. Relational Schema

2.1 Relationship Summary

#	Table	Foreign Key	References
1	**USERS**	*role_id*	ROLES(role_id)
2	**EMERGENCY_REPORTS**	*user_id*	USERS(user_id)
3	**TEAM_ASSIGNMENTS**	*report_id*	EMERGENCY_REPORTS(report_id)
4	**TEAM_ASSIGNMENTS**	*team_id*	RESCUE_TEAMS(team_id)
5	**TEAM_ASSIGNMENTS**	*assigned_by*	USERS(user_id)
6	**WAREHOUSES**	*managed_by*	USERS(user_id)
7	**RESOURCES**	*warehouse_id*	WAREHOUSES(warehouse_id)
8	**RESOURCE_ALLOCATIONS**	*resource_id*	RESOURCES(resource_id)
9	**RESOURCE_ALLOCATIONS**	*report_id*	EMERGENCY_REPORTS(report_id)
10	**RESOURCE_ALLOCATIONS**	*requested_by*	USERS(user_id)
11	**RESOURCE_ALLOCATIONS**	*approved_by*	USERS(user_id)
12	**PATIENTS**	*hospital_id*	HOSPITALS(hospital_id)
13	**PATIENTS**	*report_id*	EMERGENCY_REPORTS(report_id)
14	**DONATIONS**	*report_id*	EMERGENCY_REPORTS(report_id)
15	**DONATIONS**	*recorded_by*	USERS(user_id)
16	**EXPENSES**	*report_id*	EMERGENCY_REPORTS(report_id)
17	**EXPENSES**	*recorded_by*	USERS(user_id)
18	**EXPENSES**	*approved_by*	USERS(user_id)
19	**APPROVAL_REQUESTS**	*requested_by*	USERS(user_id)

20	**APPROVAL_REQUESTS**	*approved_by*	USERS(user_id)
21	**AUDIT_LOGS**	*user_id*	USERS(user_id)

2.2 Table Schemas

2.2.1 ROLES

Note: *Strong entity. Stores the five system roles.*

Primary Key: role_id

Attribute	Data Type	Constraints	Description
role_id	*SERIAL*	PRIMARY KEY	Auto-increment role identifier
role_name	*VARCHAR(50)*	NOT NULL, UNIQUE	Role name (Admin, Operator, etc.)
description	*TEXT*	NULL	Optional description of the role

2.2.2 USERS

Note: *Strong entity. Central to RBAC — every action traces back to a user.*

Primary Key: user_id

Foreign Keys: role_id → ROLES(role_id)

Attribute	Data Type	Constraints	Description
user_id	*SERIAL*	PRIMARY KEY	Auto-increment user identifier
username	*VARCHAR(80)*	NOT NULL, UNIQUE	Unique login username
email	*VARCHAR(120)*	NOT NULL, UNIQUE	User email address
password_hash	*VARCHAR(255)*	NOT NULL	bcrypt hashed password
role_id	*INT*	NOT NULL, FK → ROLES	Role assigned to user
created_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Account creation timestamp
is_active	*BOOLEAN*	NOT NULL, DEFAULT TRUE	Whether the account is active

2.2.3 EMERGENCY_REPORTS

Note: *Strong entity. The central hub — most other entities reference report_id.*

Primary Key: report_id

Foreign Keys: user_id → USERS(user_id)

Attribute	Data Type	Constraints	Description
report_id	*SERIAL*	PRIMARY KEY	Auto-increment report identifier
user_id	*INT*	NOT NULL, FK → USERS	User who submitted the report
location	*VARCHAR(200)*	NOT NULL	Incident location
disaster_type	*VARCHAR(80)*	NOT NULL, CHECK IN ('Flood','Earthquake','Fire','Other')	Type of disaster
severity_level	*VARCHAR(20)*	NOT NULL, CHECK IN ('Low','Medium','High','Critical')	Incident severity
description	*TEXT*	NULL	Additional details
status	*VARCHAR(30)*	NOT NULL, DEFAULT 'Open', CHECK IN ('Open','InProgress','Resolved','Closed')	Current report status
reported_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Time of report submission
updated_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Last status update time

2.2.4 RESCUE_TEAMS

Note: *Strong entity. Tracks all rescue teams and their availability.*

Primary Key: team_id

Attribute	Data Type	Constraints	Description
team_id	*SERIAL*	PRIMARY KEY	Auto-increment team identifier
team_name	*VARCHAR(100)*	NOT NULL	Name of the rescue team
team_type	*VARCHAR(30)*	NOT NULL, CHECK IN ('Medical','Fire','Rescue','Search')	Type of rescue team
current_location	*VARCHAR(200)*	NULL	Current GPS or area location

availability_status	*VARCHAR(20)*	NOT NULL, DEFAULT 'Available', CHECK IN ('Available','Assigned','Busy','Completed')	Current team status
capacity	*INT*	NOT NULL, CHECK > 0	Number of members in team
last_updated	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Last status update time

2.2.5 TEAM_ASSIGNMENTS (Weak + Associative)

Note: *Weak & Associative entity. Resolves M:N between EMERGENCY_REPORTS and RESCUE_TEAMS. Cannot exist without both parents.*

Primary Key: assignment_id (partial key)

Foreign Keys: report_id → EMERGENCY_REPORTS(report_id) | team_id → RESCUE_TEAMS(team_id) | assigned_by → USERS(user_id)

Attribute	Data Type	Constraints	Description
assignment_id	*SERIAL*	PRIMARY KEY (partial key)	Auto-increment assignment identifier
report_id	*INT*	NOT NULL, FK → EMERGENCY_REPORTS	Linked emergency report
team_id	*INT*	NOT NULL, FK → RESCUE_TEAMS	Assigned rescue team
assigned_by	*INT*	NOT NULL, FK → USERS	User who authorised the assignment
assigned_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Assignment timestamp
completed_at	*TIMESTAMP*	NULL	Completion timestamp (null if ongoing)
status	*VARCHAR(20)*	NOT NULL, DEFAULT 'Assigned', CHECK IN ('Assigned','InProgress','Completed','Cancelled')	Assignment status

2.2.6 WAREHOUSES

Note: *Strong entity. Each warehouse is managed by one user (warehouse manager role).*

Primary Key: warehouse_id

Foreign Keys: managed_by → USERS(user_id)

Attribute	Data Type	Constraints	Description
warehouse_id	*SERIAL*	PRIMARY KEY	Auto-increment warehouse identifier
warehouse_name	*VARCHAR(100)*	NOT NULL	Name of the warehouse
location	*VARCHAR(200)*	NOT NULL	Physical location of warehouse
managed_by	*INT*	NOT NULL, FK → USERS	User managing this warehouse
is_active	*BOOLEAN*	NOT NULL, DEFAULT TRUE	Whether warehouse is operational

2.2.7 RESOURCES

Note: *Strong entity. Every resource must belong to a warehouse (total participation).*

Primary Key: resource_id

Foreign Keys: warehouse_id → WAREHOUSES(warehouse_id)

Attribute	Data Type	Constraints	Description
resource_id	*SERIAL*	PRIMARY KEY	Auto-incremen t resource identifier
resource_name	*VARCHAR(100)*	NOT NULL	Descriptive name of the resource
resource_type	*VARCHAR(30)*	NOT NULL, CHECK IN ('Food','Water','Medicine','Shelter','Equipment')	Category of resource
quantity_available	*INT*	NOT NULL, DEFAULT 0, CHECK >= 0	Current stock quantity
reorder_threshold	*INT*	NOT NULL, CHECK >= 0	Low-stock alert level
warehouse_id	*INT*	NOT NULL, FK → WAREHOUSES	Warehouse storing this resource

unit	*VARCHAR(20)*	NOT NULL	Unit of measurement (kg, L, boxes)

2.2.8 RESOURCE_ALLOCATIONS (Weak + Associative)

Note: *Weak & Associative entity. Resolves M:N between RESOURCES and EMERGENCY_REPORTS.*

Primary Key: allocation_id (partial key)

Foreign Keys: resource_id → RESOURCES(resource_id) | report_id → EMERGENCY_REPORTS(report_id) | requested_by → USERS(user_id) | approved_by → USERS(user_id)

Attribute	Data Type	Constraints	Description
allocation_id	*SERIAL*	PRIMARY KEY (partial key)	Auto-increment allocation identifier
resource_id	*INT*	NOT NULL, FK → RESOURCES	Resource being allocated
report_id	*INT*	NOT NULL, FK → EMERGENCY_REPORTS	Linked emergency report
quantity_requested	*INT*	NOT NULL, CHECK > 0	Quantity requested
quantity_approved	*INT*	NULL, CHECK >= 0	Quantity approved (null if pending)
status	*VARCHAR(20)*	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending','Approved','Rejected','Dispatched','Consumed')	Allocation status
requested_by	*INT*	NOT NULL, FK → USERS	User who requested the allocation
approved_by	*INT*	NULL, FK → USERS	User who approved

			(null if pending)
requested_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Request timestamp
approved_at	*TIMESTAMP*	NULL	Approval timestamp

2.2.9 HOSPITALS

Note: *Strong entity. Stores hospital capacity for automated patient assignment.*

Primary Key: hospital_id

Attribute	Data Type	Constraints	Description
hospital_id	*SERIAL*	PRIMARY KEY	Auto-increment hospital identifier
hospital_name	*VARCHAR(150)*	NOT NULL	Full hospital name
location	*VARCHAR(200)*	NOT NULL	Hospital address / area
total_beds	*INT*	NOT NULL, CHECK > 0	Total bed capacity
available_beds	*INT*	NOT NULL, DEFAULT 0, CHECK >= 0	Currently available beds
contact_number	*VARCHAR(20)*	NULL	Emergency contact number
is_active	*BOOLEAN*	NOT NULL, DEFAULT TRUE	Whether hospital is operational

2.2.10 PATIENTS (Weak)

Note: *Weak entity. Every patient must be linked to a hospital and a report.*

Primary Key: patient_id (partial key)

Foreign Keys: hospital_id → HOSPITALS(hospital_id) | report_id → EMERGENCY_REPORTS(report_id)

Attribute	Data Type	Constraints	Description
patient_id	*SERIAL*	PRIMARY KEY (partial key)	Auto-increment patient identifier
patient_name	*VARCHAR(100)*	NOT NULL	Patient full name

age	*INT*	NULL, CHECK BETWEEN 0 AND 150	Patient age
condition_severity	*VARCHAR(20)*	NOT NULL, CHECK IN ('Stable','Critical','Deceased')	Medical condition
report_id	*INT*	NOT NULL, FK → EMERGENCY_REPORTS	Linked emergency report
hospital_id	*INT*	NOT NULL, FK → HOSPITALS	Admitted hospital
admitted_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Admission timestamp
status	*VARCHAR(20)*	NOT NULL, DEFAULT 'Admitted', CHECK IN ('Admitted','Discharged','Deceased')	Patient status

2.2.11 DONATIONS

Note: *Strong entity. report_id is nullable — donations can be general or event-specific.*

Primary Key: donation_id

Foreign Keys: report_id → EMERGENCY_REPORTS(report_id) | recorded_by → USERS(user_id)

Attribute	Data Type	Constraints	Description
donation_id	*SERIAL*	PRIMARY KEY	Auto-increment donation identifier
donor_name	*VARCHAR(150)*	NOT NULL	Name of donor
donor_type	*VARCHAR(30)*	NOT NULL, CHECK IN ('Individual','Organization','Government')	Category of donor
amount	*DECIMAL(12,2)*	NOT NULL, CHECK > 0	Donation amount
report_id	*INT*	NULL, FK → EMERGENCY_REPORTS	Linked report (optional)
recorded_by	*INT*	NOT NULL, FK → USERS	User who recorded the donation
donated_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Donation timestamp
payment_method	*VARCHAR(30)*	NULL, CHECK IN ('Cash','Bank Transfer','Online','Cheque')	Payment method

2.2.12 EXPENSES

Note: *Strong entity. report_id is nullable — expenses can be general operational costs.*

Primary Key: expense_id

Foreign Keys: report_id → EMERGENCY_REPORTS(report_id) | recorded_by → USERS(user_id) | approved_by → USERS(user_id)

Attribute	Data Type	Constraints	Description
expense_id	*SERIAL*	PRIMARY KEY	Auto-increment expense identifier
expense_category	*VARCHAR(80)*	NOT NULL	Category (Transport, Medical, etc.)
amount	*DECIMAL(12,2)*	NOT NULL, CHECK > 0	Expense amount
description	*TEXT*	NULL	Details of the expense
report_id	*INT*	NULL, FK → EMERGENCY_REPORTS	Linked report (optional)
recorded_by	*INT*	NOT NULL, FK → USERS	User who recorded the expense
approved_by	*INT*	NULL, FK → USERS	User who approved (null if pending)
expense_date	*DATE*	NOT NULL	Date of expense
status	*VARCHAR(20)*	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending','Approved','Rejected')	Approval status

2.2.13 APPROVAL_REQUESTS

Note: *Strong entity. Generic approval table covering resource allocations, team deployments, and expenses.*

Primary Key: approval_id

Foreign Keys: requested_by → USERS(user_id) | approved_by → USERS(user_id)

Attribute	Data Type	Constraints	Description
approval_id	*SERIAL*	PRIMARY KEY	Auto-increment approval identifier
request_type	*VARCHAR(40)*	NOT NULL, CHECK IN ('ResourceAllocation','TeamDeployment','Expense','Other')	What is being approved

reference_id	*INT*	NOT NULL	ID of the referenced record
requested_by	*INT*	NOT NULL, FK → USERS	User who raised the request
approved_by	*INT*	NULL, FK → USERS	User who actioned it (null if pending)
status	*VARCHAR(20)*	NOT NULL, DEFAULT 'Pending', CHECK IN ('Pending','Approved','Rejected')	Approval status
remarks	*TEXT*	NULL	Notes from approver
requested_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Request submission time
actioned_at	*TIMESTAMP*	NULL	Approval/rejection time

2.2.14 AUDIT_LOGS (Weak)

Note: *Weak entity. Every log entry must be tied to a user. Write-once — never updated.*

Primary Key: log_id (partial key)

Foreign Keys: user_id → USERS(user_id)

Attribute	Data Type	Constraints	Description
log_id	*SERIAL*	PRIMARY KEY (partial key)	Auto-increment log identifier
user_id	*INT*	NOT NULL, FK → USERS	User who performed the action
action_type	*VARCHAR(20)*	NOT NULL, CHECK IN ('INSERT','UPDATE','DELETE','LOGIN','LOGOUT')	Type of action performed
table_name	*VARCHAR(60)*	NOT NULL	Table affected by the action
record_id	*INT*	NULL	Primary key of affected record
old_value	*TEXT*	NULL	Previous value (for UPDATE/DELETE)

new_value	*TEXT*	NULL	New value (for INSERT/UPDATE)
logged_at	*TIMESTAMP*	NOT NULL, DEFAULT NOW()	Exact timestamp of action
ip_address	*VARCHAR(45)*	NULL	IP address of the user session

3. Normalization

Step 0 — Unnormalized Form (UNF)

Imagine a government analyst records all disaster-response data in a single flat spreadsheet. One row per incident, with repeated groups jammed into comma-separated cells:

report_id	location	disaster_type	severity	reported_by_name	reported_by_email	reported_by_role	teams_assigned	team_types	resources_used	warehouses	hospital_name	available_beds	donor_names	donaton_amounts	expense_items	expense_amounts
R001	Lahore	Flood	Critical	Ali Khan	ali@gov.pk	Operator	Team-A, Team-B	Medical, Fire	Food 200kg, Water 500L	WH-1, WH-2	JPMC	120	Ahmed, Red Cross	50000, 200000	Fuel, Medicine	8000, 15000
R002	Karachi	Earthquake	High	Sara Malik	sara@gov.pk	Operator	Team-C	Rescue	Medicine 100 boxes	WH-1	Aga Khan	80	Anonymous	10000	Transport	5000

Problems in UNF:

• Repeating groups: teams_assigned, donor_names, expense_items all contain comma-separated multiple values in one cell.

• Non-atomic values: 'Food 200kg, Water 500L' mixes resource type and quantity in one cell.

• No primary key uniquely identifies each meaningful fact.

• Update anomaly: changing Ali Khan's email requires finding every row where he appears.
• Deletion anomaly: deleting R002 loses Sara Malik's user record entirely.

Step 1 — First Normal Form (1NF)

Definition: A table is in 1NF if:

(a) Every column contains only atomic (indivisible) values — no lists or sets.
(b) Every column contains values of a single type.
(c) Each column has a unique name.
(d) The order of rows and columns does not matter.
(e) There is a primary key (single or composite) that uniquely identifies each row.

1NF Violations Found in UNF

Column	Violation	Example of Bad Value
teams_assigned	Repeating group — multiple teams in one cell	Team-A, Team-B
team_types	Repeating group — must match teams_assigned positionally	Medical, Fire
resources_used	Non-atomic — mixes resource name and quantity	Food 200kg, Water 500L
warehouses	Repeating group — multiple warehouses per row	WH-1, WH-2
donor_names	Repeating group — multiple donors per incident	Ahmed, Red Cross
donation_amounts	Repeating group — positionally paired with donor_names	50000, 200000
expense_items	Repeating group — multiple expenses per incident	Fuel, Medicine
expense_amounts	Repeating group — positionally paired with expense_items	8000, 15000

1NF Solution — Decompose into Atomic Tables

We eliminate all repeating groups by pulling each group into its own table, and each row now holds exactly one fact.

INCIDENT_REPORTS_1NF

report_id (PK)	location	disaster_type	severity	reported_by_name	reported_by_email	reported_by_role	status	reported_at
R001	Lahore	Flood	Critical	Ali Khan	ali@gov.pk	Operator	Open	2024-01-10 08:00
R002	Karachi	Earthquake	High	Sara Malik	sara@gov.pk	Operator	Open	2024-01-11 09:30

TEAM_ASSIGNMENT_1NF — Repeating group extracted

assignment_id (PK)	report_id	team_name	team_type	assigned_at	status
A001	R001	Team-A	Medical	2024-01-10 08:30	Completed
A002	R001	Team-B	Fire	2024-01-10 08:45	Completed
A003	R002	Team-C	Rescue	2024-01-11 10:00	InProgress

RESOURCE_USE_1NF — Repeating group extracted, values made atomic

resource_use_id (PK)	report_id	resource_name	resource_type	quantity	unit	warehouse_name
RU001	R001	Rice	Food	200	kg	WH-1
RU002	R001	Water Bottles	Water	500	L	WH-2
RU003	R002	Paracetamol	Medicine	100	boxes	WH-1

DONATION_1NF — Repeating group extracted

donation_id (PK)	report_id	donor_name	donor_type	amount	donated_at
D001	R001	Ahmed	Individual	50000	2024-01-10

D002	R001	Red Cross	Organization	200000	2024-01-10
D003	R002	Anonymous	Individual	10000	2024-01-11

EXPENSE_1NF — Repeating group extracted

expense_id (PK)	report_id	expense_item	amount	expense_date
E001	R001	Fuel	8000	2024-01-10
E002	R001	Medicine	15000	2024-01-10
E003	R002	Transport	5000	2024-01-11

All tables now satisfy 1NF: every cell is atomic, every row has a unique PK, no repeating groups.

Step 2 — Second Normal Form (2NF)

Definition: A table is in 2NF if:

(a) It is already in 1NF.

(b) Every non-key attribute is FULLY functionally dependent on the ENTIRE primary key.

(c) There are NO partial dependencies (an attribute depends on only part of a composite PK).

Note: 2NF is only relevant when the primary key is composite (more than one column).

2NF Violations Found in 1NF Tables

After 1NF, examine tables that have composite primary keys for partial dependencies.

Violation 1 — TEAM_ASSIGNMENT_1NF

Composite PK: (report_id, team_name)

Attribute	Depends On	Full or Partial?	Action Required
assigned_at	(report_id, team_name) — both needed	**Full ✔**	Keep in assignment table
status	(report_id, team_name) — both needed	**Full ✔**	Keep in assignment table
team_type	team_name only — not report_id	**Partial ✘**	Move to separate TEAM table

Fix: Create RESCUE_TEAMS table for team_name and team_type. Remove team_type from TEAM_ASSIGNMENT.

Violation 2 — RESOURCE_USE_1NF

Composite PK: (report_id, resource_name)

Attribute	Depends On	Full or Partial?	Action Required
quantity	(report_id, resource_name) — both	**Full ✔**	Keep in allocation table
resource_type	resource_name only	**Partial ✘**	Move to RESOURCES table
unit	resource_name only	**Partial ✘**	Move to RESOURCES table
warehouse_name	resource_name only (resource lives in one warehouse)	**Partial ✘**	Move to RESOURCES table

Fix: Create RESOURCES table for resource_type, unit, warehouse_name. Keep only quantity in RESOURCE_ALLOCATIONS.

Violation 3 — INCIDENT_REPORTS_1NF

Composite PK: report_id (single key — but user attributes partially depend on the reporter, not the report)

Attribute	Depends On	Full or Partial?	Action Required
location	report_id	**Full ✔**	Keep
disaster_type	report_id	**Full ✔**	Keep
severity	report_id	**Full ✔**	Keep
reported_by_name	reporter identity (not report_id)	**Partial ✘**	Move to USERS table
reported_by_email	reporter identity	**Partial ✘**	Move to USERS table
reported_by_role	reporter identity	**Partial ✘**	Move to USERS table

Fix: Create USERS table. Replace reported_by_name / email / role with user_id FK in EMERGENCY_REPORTS.

2NF Result — Tables After Fixing Partial Dependencies

EMERGENCY_REPORTS_2NF

report_id (PK)	user_id (FK)	location	disaster_type	severity	status	reported_at
R001	U001	Lahore	Flood	Critical	Open	2024-01-10 08:00
R002	U002	Karachi	Earthquake	High	Open	2024-01-11 09:30

USERS_2NF — Extracted from INCIDENT_REPORTS

user_id (PK)	username	email	role_id (FK)	is_active
U001	ali_khan	ali@gov.pk	R01	TRUE
U002	sara_malik	sara@gov.pk	R01	TRUE

RESCUE_TEAMS_2NF — Extracted from TEAM_ASSIGNMENT

team_id (PK)	team_name	team_type	availability_status	capacity
T001	Team-A	Medical	Available	10
T002	Team-B	Fire	Available	8
T003	Team-C	Rescue	Busy	12

TEAM_ASSIGNMENTS_2NF — team_type removed, FK added

assignment_id (PK)	report_id (FK)	team_id (FK)	assigned_at	status
A001	R001	T001	2024-01-10 08:30	Completed
A002	R001	T002	2024-01-10 08:45	Completed
A003	R002	T003	2024-01-11 10:00	InProgress

RESOURCES_2NF — resource_type / unit extracted from RESOURCE_USE

resource_id (PK)	resource_name	resource_type	unit	quantity_available	warehouse_id (FK)
RS001	Rice	Food	kg	2000	WH001
RS002	Water Bottles	Water	L	5000	WH002
RS003	Paracetamol	Medicine	boxes	500	WH001

All tables now satisfy 2NF: no non-key attribute depends on only part of the primary key.

Step 3 — Third Normal Form (3NF)

Definition: A table is in 3NF if:

(a) It is already in 2NF.

(b) Every non-key attribute is NON-TRANSITIVELY dependent on the primary key.

(c) No non-key attribute determines another non-key attribute.

Rule: X → Y → Z is a transitive dependency. Y must become its own table so Z depends directly on Y's PK.

3NF Violations Found in 2NF Tables

Violation 1 — USERS_2NF: role_name stored with user

Transitive dependency chain: user_id → role_id → role_name, role_description

Dependency	Type	Problem
user_id → role_id	**Direct - yes**	Fine — role_id depends on user_id directly
role_id → role_name	**Transitive - no**	role_name does not depend on user_id, it depends on role_id
role_id → role_description	**Transitive - no**	role_description depends on role_id, not user_id

Fix: Create ROLES table with role_id, role_name, description. Store only role_id FK in USERS.

Violation 2 — RESOURCES_2NF: warehouse details stored with resource

Transitive dependency chain: resource_id → warehouse_id → warehouse_name, warehouse_location, managed_by

Dependency	Type	Problem
resource_id → warehouse_id	**Direct - yes**	Fine — warehouse_id depends on resource directly
warehouse_id → warehouse_name	**Transitive - no**	warehouse_name depends on warehouse, not the resource
warehouse_id → warehouse_location	**Transitive - no**	Location of warehouse depends on warehouse, not resource
warehouse_id → managed_by	**Transitive - no**	Manager of warehouse depends on warehouse, not resource

Fix: Create WAREHOUSES table. Store only warehouse_id FK in RESOURCES.

Violation 3 — HOSPITALS implied in PATIENTS

Transitive dependency chain: patient_id → hospital_id → hospital_name, total_beds, available_beds, contact_number

Dependency	Type	Problem
patient_id → hospital_id	**Direct - yes**	Fine
hospital_id → hospital_name	**Transitive - no**	hospital_name depends on hospital, not on the patient
hospital_id → available_beds	**Transitive - no**	Bed count belongs to hospital, not to a specific patient
hospital_id → contact_number	**Transitive - no**	Contact info belongs to hospital, not to patient record

Fix: Create HOSPITALS table. Store only hospital_id FK in PATIENTS.

Violation 4 — EXPENSE / DONATION: approval / recording user details

Transitive dependency: expense_id → recorded_by → username, email, role

Dependency	Type	Problem
expense_id → recorded_by (user_id)	**Direct - yes**	Fine — recorded_by is just an FK
recorded_by → username	**Transitive - no**	Username depends on the user, not on the expense

recorded_by → email	**Transitive - no**	Email depends on the user, not on the expense

Fix: Already resolved — USERS is a separate table and EXPENSES only stores the FK (recorded_by). No duplication.

3NF Result — Final Tables After Removing Transitive Dependencies

ROLES_3NF — Extracted from USERS

role_id (PK)	role_name	description
R01	Administrator	Full system access — manage users, view all reports and finances
R02	Emergency Operator	Submit and track emergency reports, assign rescue teams
R03	Field Officer	View assigned tasks, update team status in the field
R04	Warehouse Manager	Manage inventory, approve resource allocation requests
R05	Finance Officer	Record donations and expenses, view financial summaries

USERS_3NF — role details removed, role_id FK retained

user_id (PK)	username	email	password_hash	role_id (FK)	is_active
U001	ali_khan	ali@gov.pk	$2b$10$xxx	R02	TRUE
U002	sara_malik	sara@gov.pk	$2b$10$yyy	R02	TRUE
U003	admin_user	admin@gov.pk	$2b$10$zzz	R01	TRUE

WAREHOUSES_3NF — Extracted from RESOURCES

warehouse_id (PK)	warehouse_name	location	managed_by (FK)	is_active
WH001	Central Warehouse	Lahore	U003	TRUE

WH002	Emergency Store	Karachi	U003	TRUE

RESOURCES_3NF — warehouse details removed, warehouse_id FK retained

resource_id (PK)	resource_name	resource_type	quantity_available	reorder_threshold	unit	warehouse_id (FK)
RS001	Rice	Food	2000	200	kg	WH001
RS002	Water Bottles	Water	5000	500	L	WH002
RS003	Paracetamol	Medicine	500	50	boxes	WH001

HOSPITALS_3NF — Extracted from PATIENTS

hospital_id (PK)	hospital_name	location	total_beds	available_beds	contact_number	is_active
H001	JPMC	Karachi	500	120	021-9999	TRUE
H002	Aga Khan Hospital	Karachi	600	80	021-8888	TRUE

PATIENTS_3NF — hospital details removed, hospital_id FK retained

patient_id (PK)	patient_name	age	condition_severity	report_id (FK)	hospital_id (FK)	admitted_at	status
P001	Usman Ali	34	Critical	R001	H001	2024-01-10 10:00	Admitted
P002	Fatima Bibi	60	Stable	R002	H002	2024-01-11 12:00	Discharged

All tables now satisfy 3NF: no non-key attribute determines another non-key attribute.

Final Summary — All 14 Tables After Full Normalization

#	Table Name	Normal Form	Partial Deps Removed	Transitive Deps Removed	Entity Type

1	ROLES	3NF	None (single PK)	role_name, description moved out of USERS	Strong
2	USERS	3NF	reported_by_* moved out	role_name, email moved to ROLES	Strong
3	EMERGENCY_REPORTS	3NF	user attrs moved out	No transitive deps remain	Strong
4	RESCUE_TEAMS	3NF	team_type moved out of TEAM_ASSIGNMENTS	No transitive deps remain	Strong
5	TEAM_ASSIGNMENTS	3NF	team_type removed (composite key resolved)	No transitive deps remain	Weak+Assoc
6	WAREHOUSES	3NF	None	warehouse details removed from RESOURCES	Strong
7	RESOURCES	3NF	resource_type/unit moved out	warehouse_name/location moved to WAREHOUSES	Strong
8	RESOURCE_ALLOCATIONS	3NF	qty fields kept only (composite key)	No transitive deps remain	Weak+Assoc
9	HOSPITALS	3NF	None	hospital details removed from PATIENTS	Strong
10	PATIENTS	3NF	None (composite owner)	hospital_name, beds moved to HOSPITALS	Weak
11	DONATIONS	3NF	None	donor not linked to user record (no transitive dep)	Strong
12	EXPENSES	3NF	None	approver/recorder details live in USERS	Strong
13	APPROVAL_REQUESTS	3NF	None	user details not repeated, FK only	Strong
14	AUDIT_LOGS	3NF	None	user details live in USERS, FK only	Weak

Functional Dependencies — Final 3NF State

The following functional dependencies exist in the final schema. Each non-key attribute depends directly and only on the primary key of its table:

Table	Functional Dependency (FD)	Dependency Type
ROLES	role_id → role_name, description	Direct (key → non-key)
USERS	user_id → username, email, password_hash, role_id, is_active	Direct (key → non-key)
EMERGENCY_REPORTS	report_id → user_id, location, disaster_type, severity, status	Direct (key → non-key)
RESCUE_TEAMS	team_id → team_name, team_type, availability_status, capacity	Direct (key → non-key)
TEAM_ASSIGNMENTS	assignment_id → report_id, team_id, assigned_by, status	Direct (key → non-key)
WAREHOUSES	warehouse_id → warehouse_name, location, managed_by	Direct (key → non-key)
RESOURCES	resource_id → resource_name, resource_type, quantity, warehouse_id	Direct (key → non-key)
RESOURCE_ALLOCATIONS	allocation_id → resource_id, report_id, qty_requested, status	Direct (key → non-key)
HOSPITALS	hospital_id → hospital_name, location, total_beds, available_beds	Direct (key → non-key)
PATIENTS	patient_id → patient_name, age, condition, report_id, hospital_id	Direct (key → non-key)
DONATIONS	donation_id → donor_name, amount, report_id, recorded_by	Direct (key → non-key)
EXPENSES	expense_id → category, amount, report_id, recorded_by, status	Direct (key → non-key)
APPROVAL_REQUESTS	approval_id → request_type, reference_id, requested_by, status	Direct (key → non-key)
AUDIT_LOGS	log_id → user_id, action_type, table_name, old_value, new_value	Direct (key → non-key)

Part 2: Database Design Rationale

1. Database Design Decisions

The database was designed around the operational requirements of a national disaster response system using Microsoft SQL Server. Every decision prioritised data integrity, performance under high concurrent load, and separation of concerns across 14 entities.

1.1 Why 14 Entities

Entity	Why Separate	Alternative Rejected
ROLES	Roles change independently. New roles added without touching user records.	Storing role as VARCHAR in USERS — causes redundancy, no referential integrity.
USERS	Central authentication anchor. Every action traces to a user.	Embedding user info in each table — violates 2NF, massive redundancy.
EMERGENCY_REPORTS	Primary business event. All resources, teams, patients link to a report.	One flat table — creates repeating groups, violates 1NF.
RESCUE_TEAMS	Teams exist independently. Type and capacity are intrinsic properties.	Storing team details inside TEAM_ASSIGNMENTS — partial dependency, violates 2NF.
TEAM_ASSIGNMENTS	Resolves M:N between REPORTS and RESCUE_TEAMS with assignment-specific data.	Direct FK — cannot handle one report needing multiple teams.
WAREHOUSES	Has its own location and manager independent of what it stores.	Storing warehouse details in RESOURCES — transitive dependency, violates 3NF.
RESOURCES	Has its own type, threshold, unit independent of any allocation.	Storing in RESOURCE_ALLOCATIONS — partial dependency on resource not allocation.
RESOURCE_ALLOCATIONS	Resolves M:N between RESOURCES and REPORTS with qty and status.	Direct FK — cannot handle one report needing multiple resources.
HOSPITALS	Capacity data changes independently of patients.	Storing in PATIENTS — hospital_name and beds repeat per patient (3NF violation).

PATIENTS	Linked to a report and hospital. No independent existence.	Storing in EMERGENCY_REPORTS — multiple patients per report creates repeating groups.
DONATIONS	Can be general (not tied to a report). Independent financial lifecycle.	Merging with EXPENSES — nullable columns and type discriminator anti-pattern.
EXPENSES	Requires approval chain. Different workflow from donations.	Merging with DONATIONS — ambiguous constraints, different CHECK rules.
APPROVAL_REQUESTS	Generic workflow covering three request types in one centralised table.	Three separate approval tables — identical logic duplicated across all three.
AUDIT_LOGS	Write-once high-volume. Must never be updated. Supports archival.	Storing audit data in operational tables — pollutes business tables with metadata.

1.2 SQL Server Data Type Choices

SQL Server Type	Used For	Justification
INT IDENTITY(1,1)	All primary keys	Auto-increment PK. No application-level key generation. Equivalent to PostgreSQL SERIAL.
NVARCHAR(n)	Names, types, statuses	Unicode string with bounded length. Supports Urdu/Arabic characters if needed. Prevents unbounded growth.
NVARCHAR(MAX)	description, remarks, old_value	Unbounded Unicode text for free-form fields. Replaces PostgreSQL TEXT.
BIT	is_active, Boolean flags	Two-state flag (1/0). More semantically clear than CHAR(1). Natively indexed by SQL Server.
DATETIME2	All timestamps	Higher precision than DATETIME. Supports fractional seconds. Replaces PostgreSQL TIMESTAMP.
DATE	expense_date	Date-only. Time of day irrelevant for financial reporting. Same as PostgreSQL DATE.
DECIMAL(12,2)	amount in DONATIONS, EXPENSES	Fixed-point arithmetic. Avoids floating-point rounding errors. Same as PostgreSQL.
INT	Foreign keys, quantities	4-byte integer matching IDENTITY PKs. Efficient join performance.

1.3 Constraint Strategy

Constraint	Example	Business Rule Enforced
PRIMARY KEY	report_id, user_id	Every row uniquely identifiable. SQL Server auto-creates clustered index on PK.
FOREIGN KEY	USERS.role_id → ROLES(role_id)	Referential integrity. Cannot assign user to non-existent role.
NOT NULL	username, location	Critical fields always have values. No silent missing data.
UNIQUE	username, email	No duplicate logins or emails. SQL Server creates non-clustered index automatically.
CHECK (IN)	disaster_type IN ('Flood','Earthquake',...)	Only valid enum values accepted. Enforced at DB level regardless of application.
CHECK (>0)	capacity > 0, amount > 0	Quantities must be positive. Zero-capacity teams and negative amounts blocked.
CHECK (>=0)	quantity_available >= 0	Stock cannot go negative at constraint level (trigger provides pre-check).
DEFAULT	status DEFAULT 'Open', DEFAULT GETDATE()	Sensible starting state. Application does not need to supply every field.

2. Choice of Entities and Relationships

2.1 Strong vs Weak Entity Classification

Entity	Type	Reason
ROLES	Strong	Exists independently. Users are assigned to it.
USERS	Strong	Exists independently. All actions trace back to a user.
EMERGENCY_REPORTS	Strong	Primary business event. Teams and resources assigned to it.
RESCUE_TEAMS	Strong	Exists before being assigned to any report.
HOSPITALS	Strong	Exists with bed capacity before any disaster occurs.
WAREHOUSES	Strong	Exists with location before storing any resource.

RESOURCES	Strong	Has name, type, quantity before ever being allocated.
DONATIONS	Strong	Can exist without a specific report (general donations).
EXPENSES	Strong	Can be a general operational cost, not tied to a report.
APPROVAL_REQUESTS	Strong	Raised by a user, exists independently of outcome.
TEAM_ASSIGNMENTS	Weak + Associative	Cannot exist without EMERGENCY_REPORTS and RESCUE_TEAMS. Resolves M:N.
RESOURCE_ALLOCATIONS	Weak + Associative	Cannot exist without RESOURCES and EMERGENCY_REPORTS. Resolves M:N.
PATIENTS	Weak	Only exists because of a disaster report and hospital admission.
AUDIT_LOGS	Weak	Only exists because a user performed an action.

2.2 Relationship Cardinality

Relationship	Cardinality	Justification
ROLES → USERS	1 : N	One role assigned to many users. Each user has exactly one role.
USERS → EMERGENCY_REPORTS	1 : N	One user submits many reports. Each report has one submitter.
EMERGENCY_REPORTS ↔ RESCUE_TEAMS	M : N	Resolved via TEAM_ASSIGNMENTS. One report needs multiple teams; one team serves multiple reports.
EMERGENCY_REPORTS ↔ RESOURCES	M : N	Resolved via RESOURCE_ALLOCATIONS. One report needs multiple resources; one resource allocated to multiple reports.
WAREHOUSES → RESOURCES	1 : N	One warehouse stores many resources. Each resource belongs to one warehouse.
HOSPITALS → PATIENTS	1 : N	One hospital admits many patients. Each patient admitted to one hospital.
USERS → AUDIT_LOGS	1 : N	Every user generates many log entries. Each log attributed to one user.

3. Transaction Handling Approach

SQL Server uses explicit transactions with BEGIN TRANSACTION, COMMIT TRANSACTION, ROLLBACK TRANSACTION, and TRY...CATCH blocks. All multi-step operations are wrapped in transactions to guarantee ACID properties.

3.1 ACID Properties in SQL Server

Property	Definition	SQL Server Implementation
Atomicity	All steps succeed or none do.	BEGIN TRANSACTION with TRY/CATCH. Any error triggers ROLLBACK TRANSACTION, undoing all steps.
Consistency	Database moves between valid states only.	CHECK constraints, NOT NULL, FK constraints enforced at engine level. Triggers enforce business rules before commit.
Isolation	Concurrent transactions do not see partial results.	SQL Server default READ COMMITTED isolation. Row-level locking used in stock deduction to prevent race conditions.
Durability	Committed data survives failures.	SQL Server Write-Ahead Log (WAL/LDF file) ensures committed data is persisted before confirmation.

3.2 Transaction Boundaries — 5 Identified Operations

Transaction	Steps	Why Atomic
Resource Allocation Approval	1. UPDATE RESOURCE_ALLOCATIONS status to Approved 2. Trigger deducts stock from RESOURCES 3. INSERT into AUDIT_LOGS	If stock deduction fails after approval is recorded, inventory becomes inconsistent.
Team Assignment	1. Check team availability 2. INSERT into TEAM_ASSIGNMENTS 3. Trigger updates RESCUE_TEAMS status	Team cannot appear Available if assigned. Assignment and status must be atomic.
Expense Approval	1. UPDATE EXPENSES status 2. UPDATE APPROVAL_REQUESTS status 3. INSERT into AUDIT_LOGS	Approval record and expense status must always match. Partial update leaves system inconsistent.
Patient Admission	1. Check hospital bed count 2. INSERT into PATIENTS 3. Trigger decrements available_beds	If patient is inserted but beds not decremented, hospital appears to have more capacity than it does.
Financial Entry + Audit	1. INSERT into DONATIONS or EXPENSES 2. INSERT into AUDIT_LOGS	Financial records must always have an audit trail. Entry without log cannot be audited for compliance.

3.3 SQL Server TRY...CATCH Pattern Used

Every transaction follows this pattern — BEGIN TRANSACTION appears before BEGIN TRY. This is required by SQL Server syntax rules:

```
BEGIN TRANSACTION;  BEGIN TRY  ...operations...  COMMIT TRANSACTION;  END TRY  BEGIN CATCH  ROLLBACK TRANSACTION;  PRINT ERROR_MESSAGE();  END CATCH;
```

4. RBAC Implementation Strategy

Role-Based Access Control uses a two-layer approach: SQL Server views restrict data visibility at the database level, and Express.js middleware enforces operation permissions at the API level.

4.1 Role Permission Matrix

Role	READ	INSERT	UPDATE	Blocked From
Administrator	All tables	Users, Roles	All tables	Cannot delete audit logs
Emergency Operator	Reports, Teams, Resources, Hospitals	Reports, Assignments, Allocations	Report status	Financial data
Field Officer	Assigned reports and teams only	Status updates only	Team assignment status	Financial, patient data
Warehouse Manager	Resources, Warehouses, Allocations	Resources	Stock quantities	Financial, patient data
Finance Officer	Donations, Expenses, Approvals	Donations, Expenses	Expense status	Operational, patient data

4.2 Implementation Layers

Layer	Technology	How It Works

Layer 1 — Data	SQL Server Views	v_pending_incidents, v_team_availability, v_warehouse_stock, v_financial_summary. Each view exposes only the columns and rows that role is permitted to see.
Layer 2 — API	Express middleware (rbac.js)	allowRoles() middleware checks req.user.role_name on every route. Returns 403 Forbidden before any SQL reaches the database.
Layer 3 — Auth	JWT (jsonwebtoken)	Role name embedded in JWT payload at login. Token verified on every request by auth.js middleware. 24-hour expiry.

5. Trigger Implementation and Automation

SQL Server triggers use INSERTED and DELETED virtual tables instead of PostgreSQL NEW/OLD. All triggers use AFTER (not INSTEAD OF) and SET NOCOUNT ON to prevent extra rowcount messages affecting the application.

5.1 Trigger Catalogue

Trigger	Event	Table	Action	Business Rule
trg_deduct_resource_stock	AFTER UPDATE	RESOURCE_ALLOCATIONS	Deducts quantity_approved from RESOURCES when status changes Pending→Approved	Stock must decrease atomically with approval. Uses INSERTED/DELETED to detect status change.
trg_block_negative_stock	AFTER UPDATE	RESOURCES	RAISERROR and ROLLBACK if quantity_available < 0	Last line of defence. Stock can never go negative even via direct SQL.
trg_update_team_status	AFTER INSERT,	TEAM_ASSIGNMENTS	Sets RESCUE_TEAMS.availability_status to Assigned or Available	Team availability must reflect actual assignment

	UPDATE			state in real time.
trg_log_emergency_report	AFTER INSERT	EMERGENCY_REPORTS	INSERT row into AUDIT_LOGS	Every report submission logged. Cannot be skipped by application.
trg_log_donation	AFTER INSERT	DONATIONS	INSERT row into AUDIT_LOGS	Financial audit trail mandatory for compliance.
trg_log_expense	AFTER INSERT	EXPENSES	INSERT row into AUDIT_LOGS	Same — every financial entry must produce an audit record.
trg_update_report_timestamp	AFTER UPDATE	EMERGENCY_REPORTS	Sets updated_at = GETDATE()	updated_at always reflects true last-modified time.
trg_update_hospital_beds	AFTER INSERT, UPDATE	PATIENTS	Decrements or increments HOSPITALS.available_beds	Bed count must stay accurate in real time for patient assignment decisions.

5.2 SQL Server Trigger Syntax

Concept	SQL Server
Row reference	INSERTED.column / DELETED.column (virtual tables)
Trigger function	Single CREATE TRIGGER block — no separate function needed
Raise error	RAISERROR('message', 16, 1) then ROLLBACK TRANSACTION
Prevent extra rowcounts	SET NOCOUNT ON at start of every trigger body
Multiple events	AFTER INSERT, UPDATE in one trigger

6. Views — Abstraction, Security, Performance

6.1 View Definitions and Purpose

View	Intended Role	Tables Joined	Purpose
v_pending_incidents	Emergency Operator	EMERGENCY_REPORTS, USERS	Shows only Open/InProgress reports. Hides Resolved/Closed. Sorts by severity automatically.
v_team_availability	Field Officer	RESCUE_TEAMS	Shows only Available teams. Hides Assigned/Busy teams and capacity planning data.
v_warehouse_stock	Warehouse Manager	WAREHOUSES, RESOURCES	Stock levels with LOW STOCK / OK alert flag. Hides financial and patient data.
v_financial_summary	Finance Officer	EMERGENCY_REPORTS, DONATIONS, EXPENSES	Aggregates donations vs expenses per disaster using ISNULL for null-safe arithmetic.
v_pending_approvals	Administrator	APPROVAL_REQUESTS, USERS	All pending approvals with requester username. Centralised approval dashboard.
v_audit_trail	Administrator	AUDIT_LOGS, USERS	Full audit trail with human-readable usernames instead of raw user_id integers.

6.2 Security Benefits

Views implement column-level and row-level security without SQL Server row security policies. A Finance Officer querying v_financial_summary cannot see patient names, team assignments, or resource quantities — those tables are simply not in the view definition. This is simpler and more portable than row-level security policies.

6.3 Performance Comparison — Views vs Direct Queries

SQL Server inlines view definitions into the query plan (similar to PostgreSQL). The query optimiser expands the view before planning, so in most cases views add zero overhead.

Query	Without View	With View	Plan Difference	Verdict
Pending incidents by severity	Clustered Index Scan + Nested Loop JOIN + CASE sort	SELECT * FROM v_pending_incidents	Identical execution plan — SQL Server inlines the view.	Same performance. View adds no cost.
Financial summary per event	Hash Match JOIN + Stream Aggregate with ISNULL	SELECT * FROM v_financial_summary	Identical plan. ISNULL handled identically.	Same performance. Abstraction is free.
Low-stock resources	Index Seek on quantity_available + reorder_threshold	SELECT * FROM v_warehouse_stock WHERE stock_alert='LOW STOCK'	View version slower — stock_alert is a computed CASE column, cannot use index. Direct query filters on indexed columns.	Direct query faster here. View trades performance for readability.
Audit trail with username	Nested Loop JOIN AUDIT_LOGS + USERS	SELECT * FROM v_audit_trail	Identical plan. Simple join abstraction.	Same performance.

Key Finding: The one exception where direct query outperforms the view is v_warehouse_stock where the computed stock_alert column prevents index usage. In production, a computed column with a persisted index would resolve this.

7. Indexing Strategy and Performance

7.1 Index Strategy

SQL Server automatically creates a clustered index on every primary key (INT IDENTITY). All additional indexes created are non-clustered. Indexes were placed on columns appearing in WHERE clauses, JOIN conditions, and ORDER BY expressions of the most frequent queries.

7.2 Index Catalogue — 24 Indexes Total

Index	Table	Columns	Type	Query Supported
PK clustered (auto)	All 14 tables	Primary key	Clustered	All PK lookups and JOIN operations
idx_reports_location	EMERGENCY_REPORTS	location	Non-clustered	Filter by city/area
idx_reports_disaster_type	EMERGENCY_REPORTS	disaster_type	Non-clustered	Filter by disaster category
idx_reports_severity	EMERGENCY_REPORTS	severity_level	Non-clustered	Prioritise critical incidents
idx_reports_status	EMERGENCY_REPORTS	status	Non-clustered	Find open/in-progress reports
idx_reports_reported_at	EMERGENCY_REPORTS	reported_at	Non-clustered	Date-range and dashboard queries
idx_reports_type_severity	EMERGENCY_REPORTS	(disaster_type, severity_level)	Composite	Combined filter: Critical Floods
idx_reports_location_type	EMERGENCY_REPORTS	(location, disaster_type)	Composite	Location + type drill-down
idx_resources_type	RESOURCES	resource_type	Non-clustered	Filter by Food/Medicine/Water
idx_resources_warehouse	RESOURCES	warehouse_id	Non-clustered	All resources in a warehouse
idx_team_status	RESCUE_TEAMS	availability_status	Non-clustered	Find available teams for dispatch
idx_allocations_status	RESOURCE_ALLOCATIONS	status	Non-clustered	Pending allocation queries
idx_alloc_resource_report	RESOURCE_ALLOCATIONS	(resource_id, report_id)	Composite	Check if resource already allocated to report
idx_donations_at	DONATIONS	donated_at	Non-clustered	Financial reports by date range
idx_expenses_date	EXPENSES	expense_date	Non-clustered	Monthly expense summaries
idx_expenses_status	EXPENSES	status	Non-clustered	Pending expense approvals

idx_audit_logged_at	AUDIT_LOGS	logged_at	Non-clustered	Recent activity queries
idx_audit_user	AUDIT_LOGS	user_id	Non-clustered	All logs for a specific user
idx_audit_user_action	AUDIT_LOGS	(user_id, action_type)	Composite	User activity by action type
idx_audit_table	AUDIT_LOGS	table_name	Non-clustered	All changes to a specific table
idx_approval_status	APPROVAL_REQUESTS	status	Non-clustered	Dashboard: pending approvals count
idx_patients_hospital	PATIENTS	hospital_id	Non-clustered	All patients in a hospital
idx_patients_report	PATIENTS	report_id	Non-clustered	All patients from a report

7.3 Performance Analysis — SET STATISTICS TIME ON

SQL Server uses SET STATISTICS TIME ON and SET STATISTICS IO ON to measure query performance (equivalent to PostgreSQL EXPLAIN ANALYZE). After running these in SSMS, the Messages tab shows CPU time and elapsed time in milliseconds.

Query	Plan Without Index	Plan With Index	Expected Improvement at Scale
disaster_type='Flood' AND severity='Critical'	Table Scan — reads all rows sequentially	Index Seek on idx_reports_type_severity — reads only matching rows	O(n) → O(log n). ~100x faster on 1M rows.
AUDIT_LOGS WHERE logged_at >= last 7 days	Table Scan on full audit table	Range Seek on idx_audit_logged_at	Critical — audit tables grow fastest in the system.
RESCUE_TEAMS WHERE availability='Available'	Table Scan on RESCUE_TEAMS	Index Seek on idx_team_status	Small table but time-critical in emergency dispatch.

RESOURCE_ALLOCATIONS JOIN on resource_id	Hash Join with full scan	Nested Loop using idx_alloc_resource_report	Eliminates full scan per resource in allocation checks.
EXPENSES WHERE expense_date in range	Table Scan	Range Seek on idx_expenses_date	Monthly financial reports run instantly.

7.4 Read vs Write Trade-offs

Table	Index Count	Write Frequency	Read Frequency	Decision
EMERGENCY_REPORTS	7 indexes	Moderate — new reports during disasters only	Very High — dashboards, filtering, joins	Read-heavy. Index maintenance cost on writes acceptable.
AUDIT_LOGS	4 indexes	Very High — every system action writes a log	Low — compliance queries only	Write-heavy. Kept only 4 essential indexes. More would slow every system operation.
RESOURCES	2 indexes	Low — stock changes per allocation approval	High — inventory dashboard, low-stock alerts	Low writes, high reads. Both indexes fully justified.
RESCUE_TEAMS	1 index	Low — status updates on assignment	High — dispatch queries every few minutes	Small table. Availability index essential for real-time dispatch.
DONATIONS / EXPENSES	2 each	Low — financial entries per event	Moderate — monthly reports, audit	Low write cost makes index maintenance negligible.

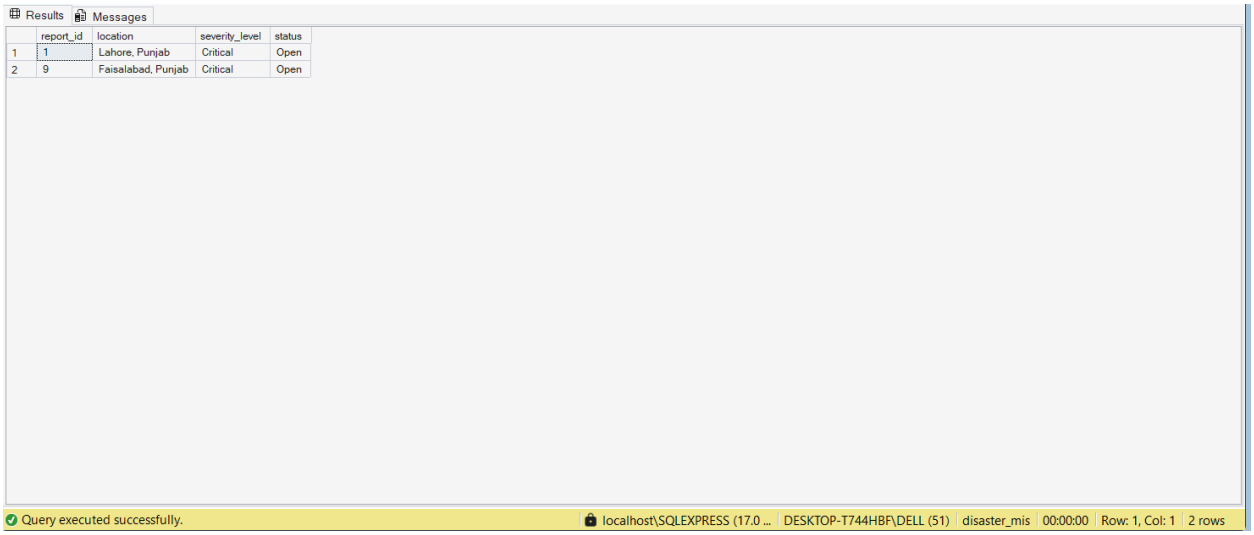
8. Summary of All Design Decisions

Design Area	Decision	Core Justification
DBMS Choice	Microsoft SQL Server Express	Course requirement. SQL Server provides enterprise-grade ACID compliance, row-level locking, clustered indexes, and native TRY/CATCH transaction handling.
Entity Count	14 entities	Each entity has a distinct lifecycle. Merging causes redundancy; splitting adds unnecessary joins.
Normalization	3NF across all 14 tables	Eliminates all update, insertion, deletion anomalies while keeping query complexity manageable.
Weak Entities	4 weak entities	TEAM_ASSIGNMENTS, RESOURCE_ALLOCATIONS, PATIENTS, AUDIT_LOGS cannot exist without owner entities.
Transactions	5 explicit transaction types with TRY/CATCH	All multi-step operations wrapped in BEGIN TRANSACTION/COMMIT/ROLLBACK. Follows SQL Server syntax rules.
RBAC	Role as FK + API middleware + Views	Three-layer enforcement. Role stored efficiently as INT FK, API checks role on every request, views restrict data visibility.
Triggers	8 triggers using INSERTED/DELETED tables	Critical business rules enforced at database level. Cannot be bypassed via SSMS or direct SQL.
Views	6 role-specific views using ISNULL	Each role sees only relevant data. ISNULL used instead of PostgreSQL COALESCE for null-safe arithmetic.
Indexes	24 indexes (14 clustered PKs + 10 non-clustered + composite)	High-read columns indexed aggressively. High-write tables (AUDIT_LOGS) indexed conservatively.
Data Types	NVARCHAR, DATETIME2, INT IDENTITY, DECIMAL, BIT	Each chosen for correctness: Unicode support, high timestamp precision, no floating-point money errors.
Constraints	CHECK, NOT NULL, FK, UNIQUE at DB level	Database enforces rules independently of application. Data integrity guaranteed even under direct SSMS access.

This design rationale demonstrates that every decision in the Smart Disaster Response MIS was made deliberately, with reference to database theory (normalization, ACID, RBAC), SQL Server-specific features (INSERTED/DELETED tables, SET STATISTICS, TRY/CATCH), and the practical constraints of a high-volume emergency response system.

9. Performance Analysis

- **Test 1**
  Query WITHOUT composite index
  With indexes



- **Test 2**
  Location search WITHOUT index
  With index

Results Messages

	report_id	location	severity_level	status
1	1	Lahore, Punjab	Critical	Open
2	9	Faisalabad, Punjab	Critical	Open

	report_id	location	disaster_type	severity_level	status	reported_at	username
1	1	Lahore, Punjab	Flood	Critical	Open	2026-05-04 14:13:12.0366667	ali_operator
2	2	Karachi, Sindh	Earthquake	High	InProgress	2026-05-04 14:13:12.0366667	ali_operator
3	3	Peshawar, KPK	Fire	High	Open	2026-05-04 14:13:12.0366667	sara_operator
4	4	Quetta, Balochistan	Earthquake	Critical	InProgress	2026-05-04 14:13:12.0366667	sara_operator
5	5	Multan, Punjab	Flood	Medium	Open	2026-05-04 14:13:12.0366667	ali_operator
6	7	Hyderabad, Sindh	Flood	High	Open	2026-05-04 14:13:12.0366667	ali_operator
7	8	Rawalpindi, Punjab	Other	Medium	InProgress	2026-05-04 14:13:12.0366667	sara_operator
8	9	Faisalabad, Punjab	Flood	Critical	Open	2026-05-04 14:13:12.0366667	ali_operator
9	10	Sukkur, Sindh	Earthquake	High	Open	2026-05-04 14:13:12.0366667	sara_operator

	report_id	location	disaster_type	severity_level	status	reported_at	reported_by
1	1	Lahore, Punjab	Flood	Critical	Open	2026-05-04 14:13:12.0366667	ali_operator
2	2	Karachi, Sindh	Earthquake	High	InProgress	2026-05-04 14:13:12.0366667	ali_operator
3	3	Peshawar, KPK	Fire	High	Open	2026-05-04 14:13:12.0366667	sara_operator
4	4	Quetta, Balochistan	Earthquake	Critical	InProgress	2026-05-04 14:13:12.0366667	sara_operator
5	5	Multan, Punjab	Flood	Medium	Open	2026-05-04 14:13:12.0366667	ali_operator
6	7	Hyderabad, Sindh	Flood	High	Open	2026-05-04 14:13:12.0366667	ali_operator
7	8	Rawalpindi, Punjab	Other	Medium	InProgress	2026-05-04 14:13:12.0366667	sara_operator
8	9	Faisalabad, Punjab	Flood	Critical	Open	2026-05-04 14:13:12.0366667	ali_operator
9	10	Sukkur, Sindh	Earthquake	High	Open	2026-05-04 14:13:12.0366667	sara_operator

Query executed successfully. localhost\SQLEXPRESS (17.0 ... DESKTOP-T744HBF\DELL (51) disaster_mis 00:00:00 Row: 1, Col: 1 20 rows

- **Test 3**
  Audit log timestamp search without indexes
  With indexes

Results Messages

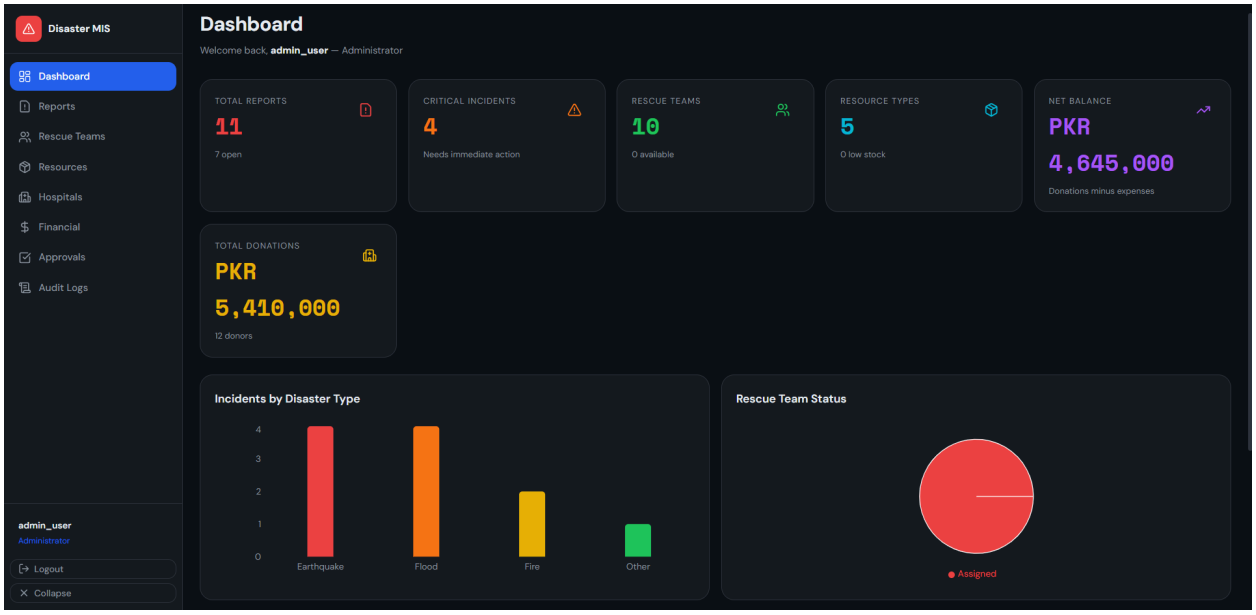
	log_id	user_id	action_type	logged_at
1	1	3	INSERT	2026-05-04 14:13:12.0533333
2	2	2	INSERT	2026-05-04 14:13:12.0533333
3	3	3	INSERT	2026-05-04 14:13:12.0533333
4	4	2	INSERT	2026-05-04 14:13:12.0533333
5	5	3	INSERT	2026-05-04 14:13:12.0533333
6	6	2	INSERT	2026-05-04 14:13:12.0533333
7	7	3	INSERT	2026-05-04 14:13:12.0533333
8	8	3	INSERT	2026-05-04 14:13:12.0533333

	log_id	user_id	action_type	logged_at
1	1	3	INSERT	2026-05-04 14:13:12.0533333
2	2	2	INSERT	2026-05-04 14:13:12.0533333
3	3	3	INSERT	2026-05-04 14:13:12.0533333
4	4	2	INSERT	2026-05-04 14:13:12.0533333
5	5	3	INSERT	2026-05-04 14:13:12.0533333
6	6	2	INSERT	2026-05-04 14:13:12.0533333
7	7	3	INSERT	2026-05-04 14:13:12.0533333
8	8	3	INSERT	2026-05-04 14:13:12.0533333
9	9	2	INSERT	2026-05-04 14:13:12.0533333
10	10	2	INSERT	2026-05-04 14:13:12.0533333
11	11	9	INSERT	2026-05-04 14:13:12.4300000
12	12	8	INSERT	2026-05-04 14:13:12.4300000
13	13	9	INSERT	2026-05-04 14:13:12.4300000
14	14	8	INSERT	2026-05-04 14:13:12.4300000
15	15	9	INSERT	2026-05-04 14:13:12.4300000

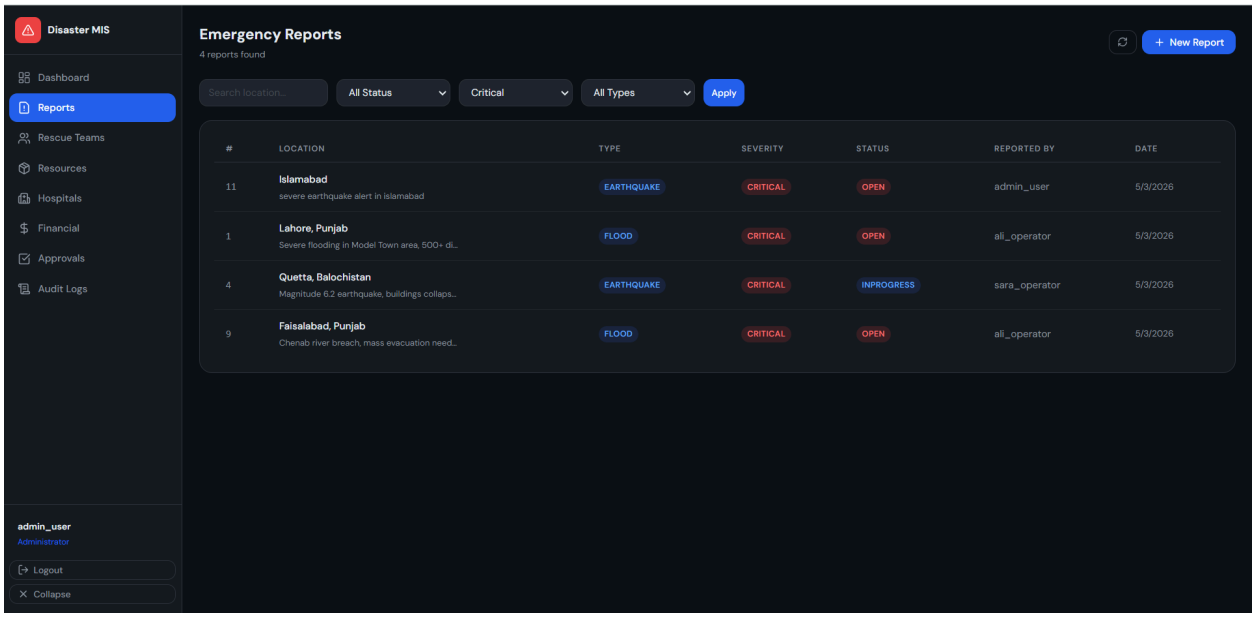
Query executed successfully. localhost\SQLEXPRESS (17.0 ... DESKTOP-T744HBF\DELL (51) disaster_mis 00:00:00 Row: 1, Col: 1 62 rows

10. MIS Reports

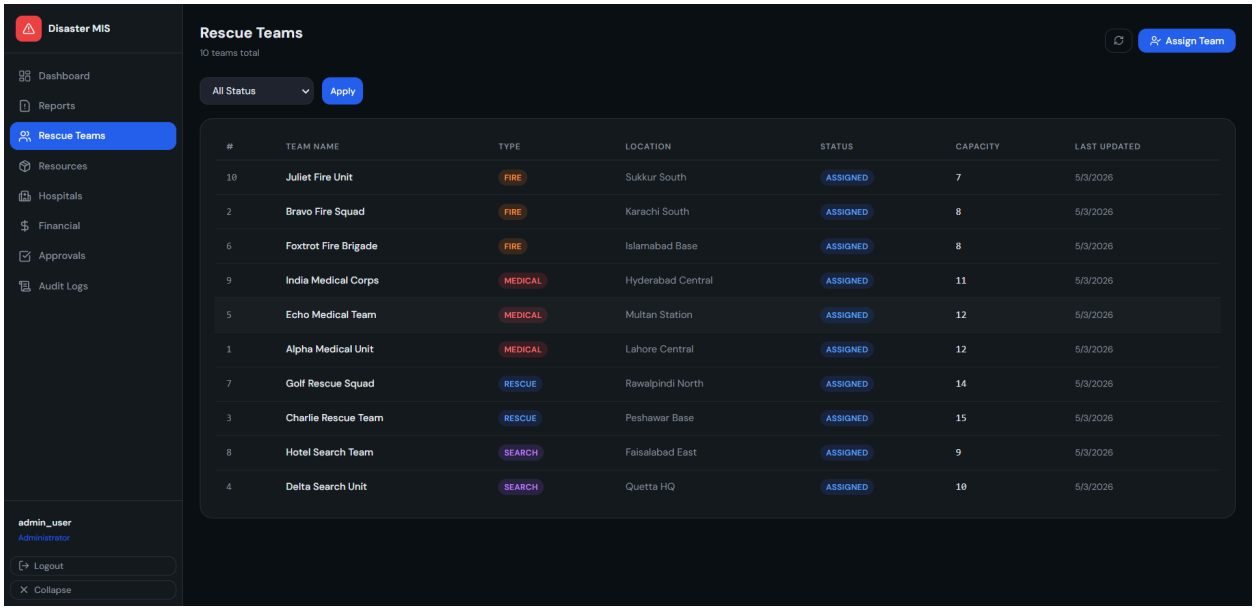
- Dashboard with charts



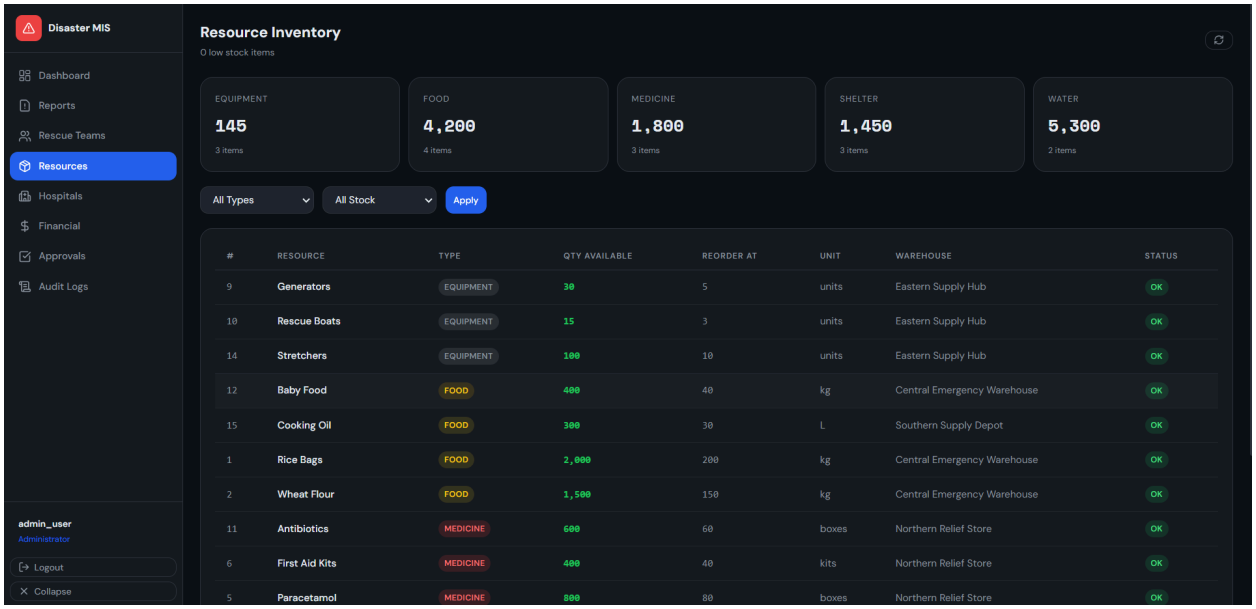
- Reports table filtered by Critical severity



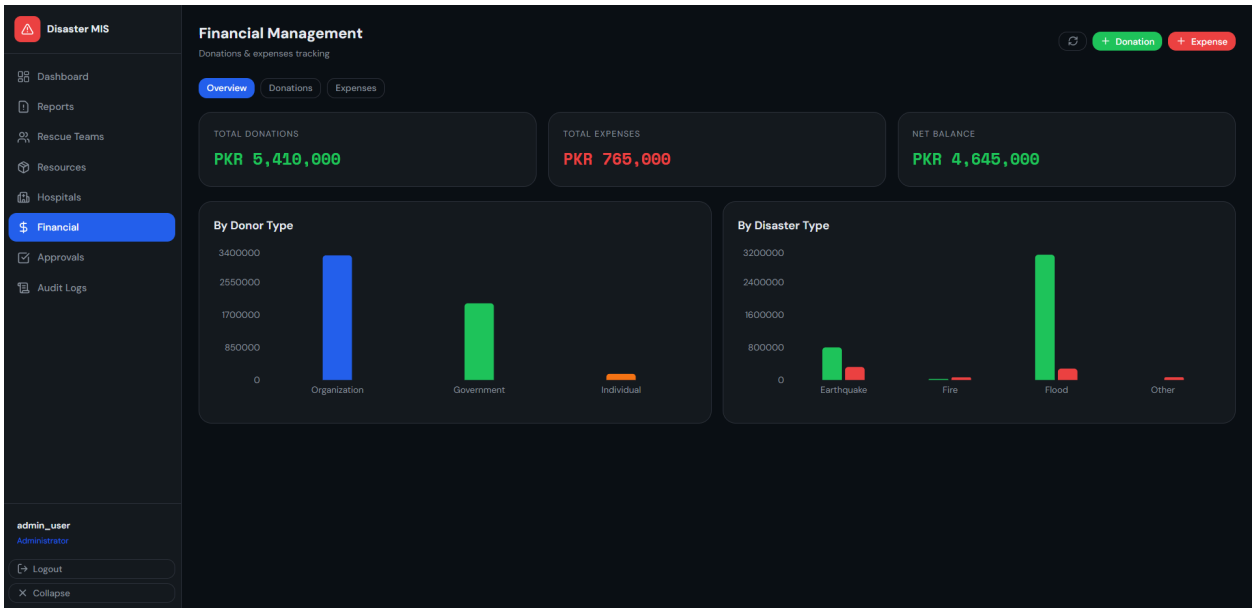
- Teams status view



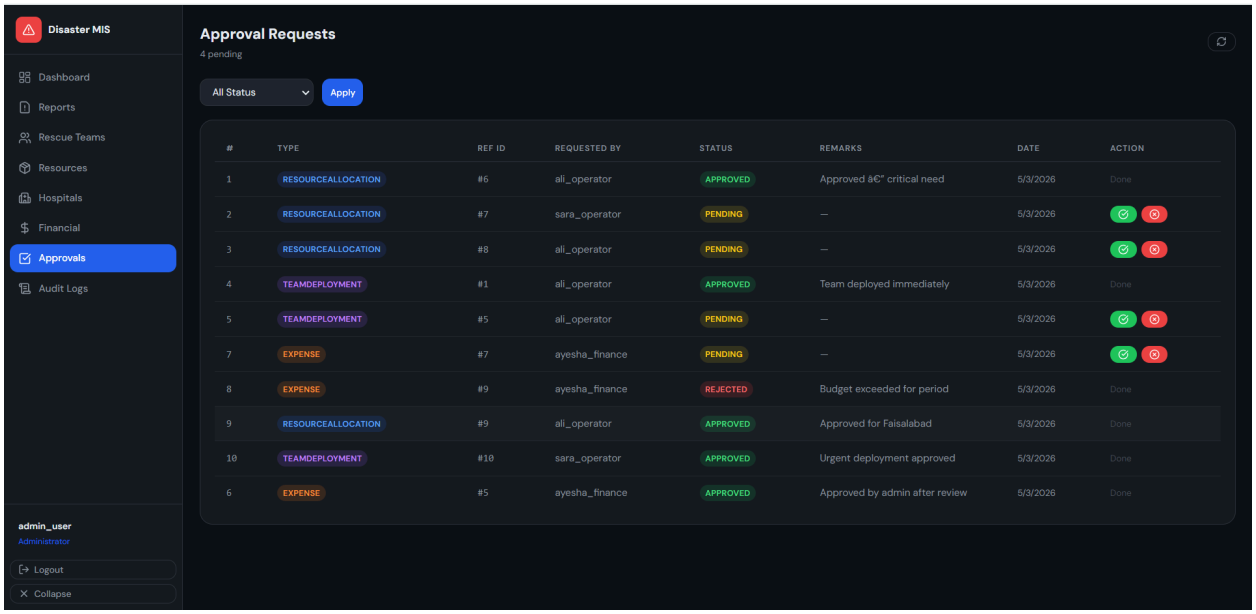
- All stock resources



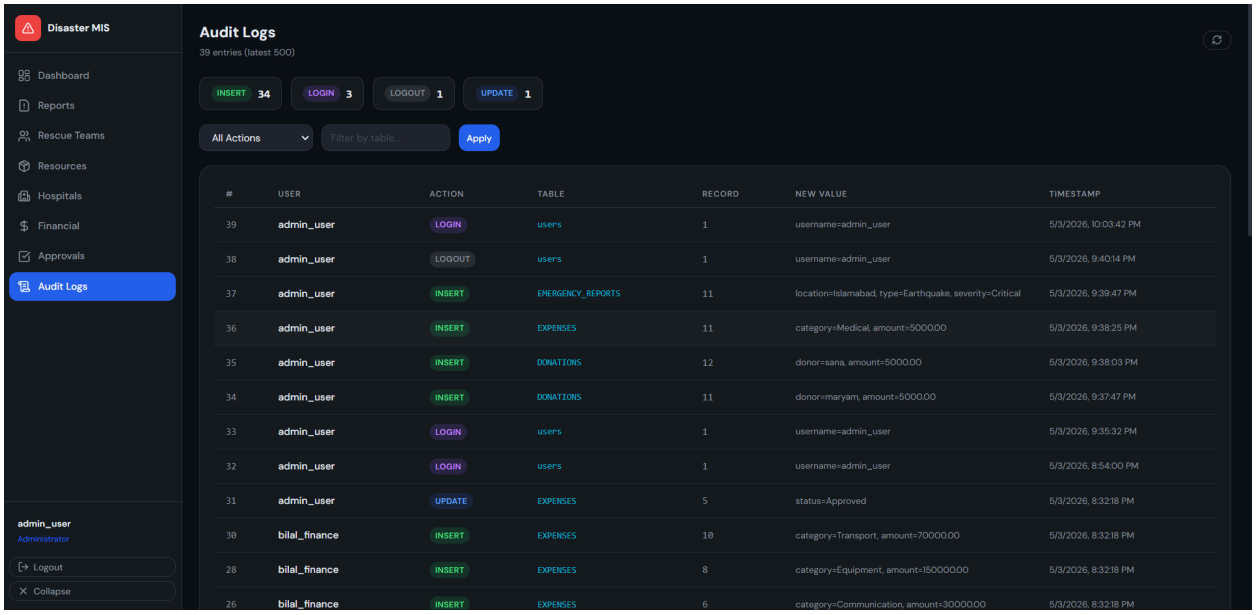
- Financial summary with charts



- Approvals list



- Audit logs table



- Hospital capacity cards